Arquitectura de deposito-service

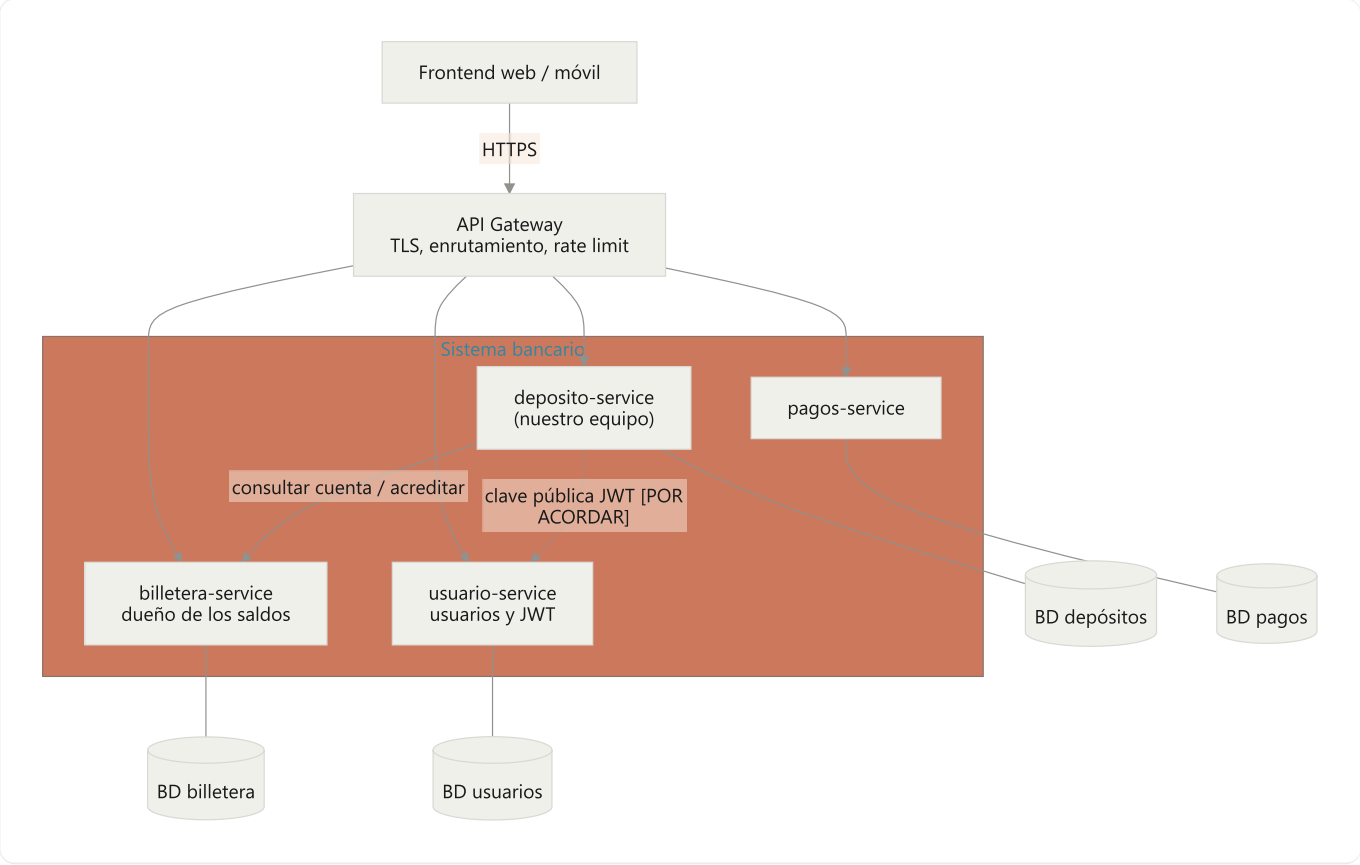
> Documento vivo. Los diagramas están en **Mermaid**: se editan como texto y se ven como gráfico en GitHub, GitLab, VS Code (extensión *Markdown Preview Mermaid Support*) u Obsidian. También se pueden pegar en https://mermaid.live para verlos al instante. Convención: lo marcado con `[POR ACORDAR]` depende de otros equipos.

Campo	Valor
Versión	0.1
Stack	Node.js (Express) + PostgreSQL
Arquitectura interna	Capas + core con puntos de extensión (módulos enchufables)
Responsable	Equipo depósito

---

1. Contexto: los 4 microservicios

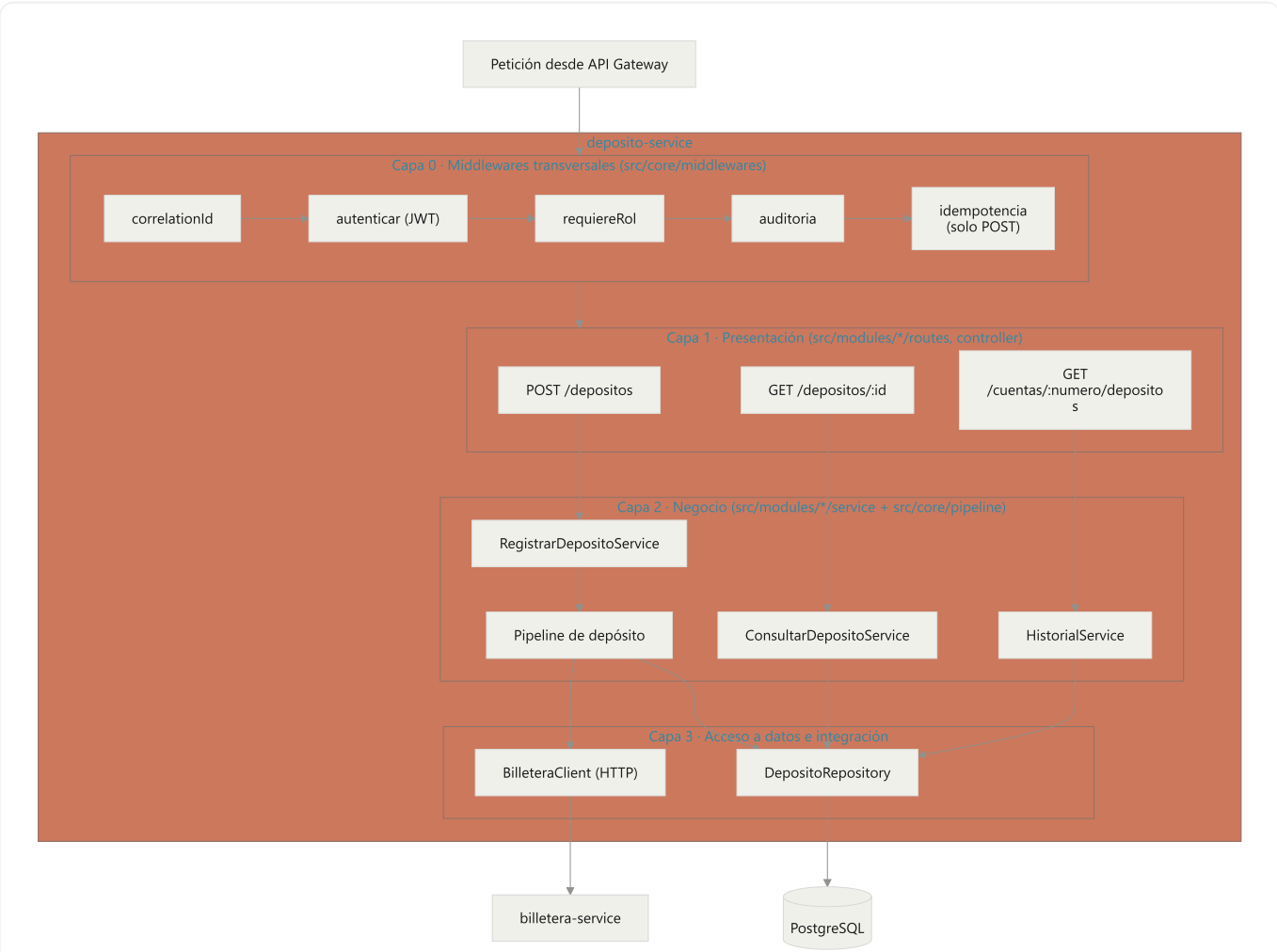
Vista general del banco. deposito-service solo habla por HTTP; nunca entra a la base de datos de otro servicio.



---

2. Arquitectura en capas (detalle interno)

Una petición baja siempre en este orden. Ninguna capa se salta a otra.



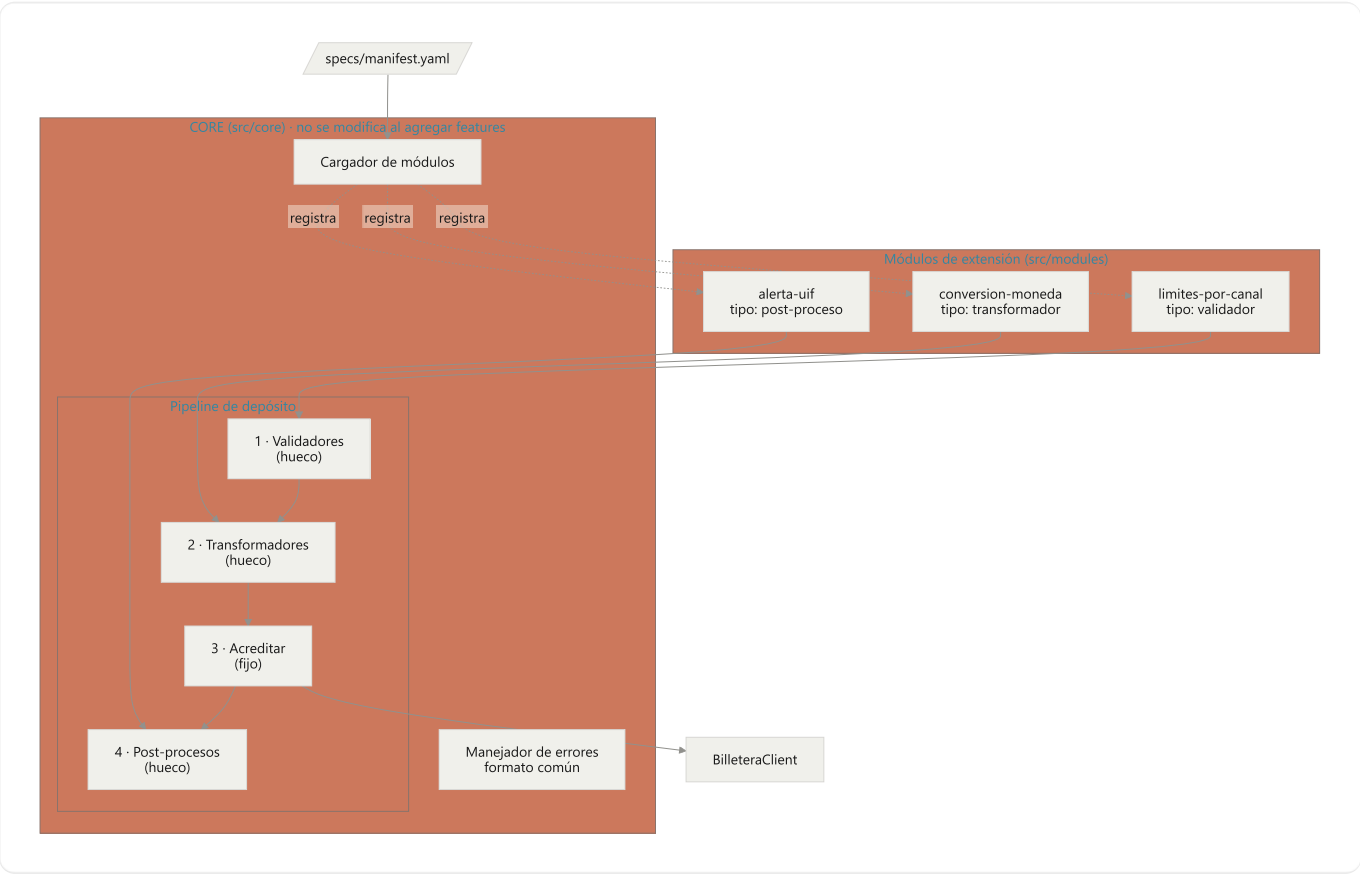
Reglas de las capas

- Los controllers no tienen lógica de negocio: leen la petición, llaman al service y responden.
- Los services no escriben SQL ni hacen llamadas HTTP directas: usan repository y client.
- Solo la capa 3 conoce PostgreSQL y las URLs de otros servicios.

---

3. Componentes: core, pipeline y módulos

El `Cargador` lee `manifest.yaml` al arrancar y enchufa los módulos activos en los huecos del pipeline. Quitar un módulo no modifica el core.

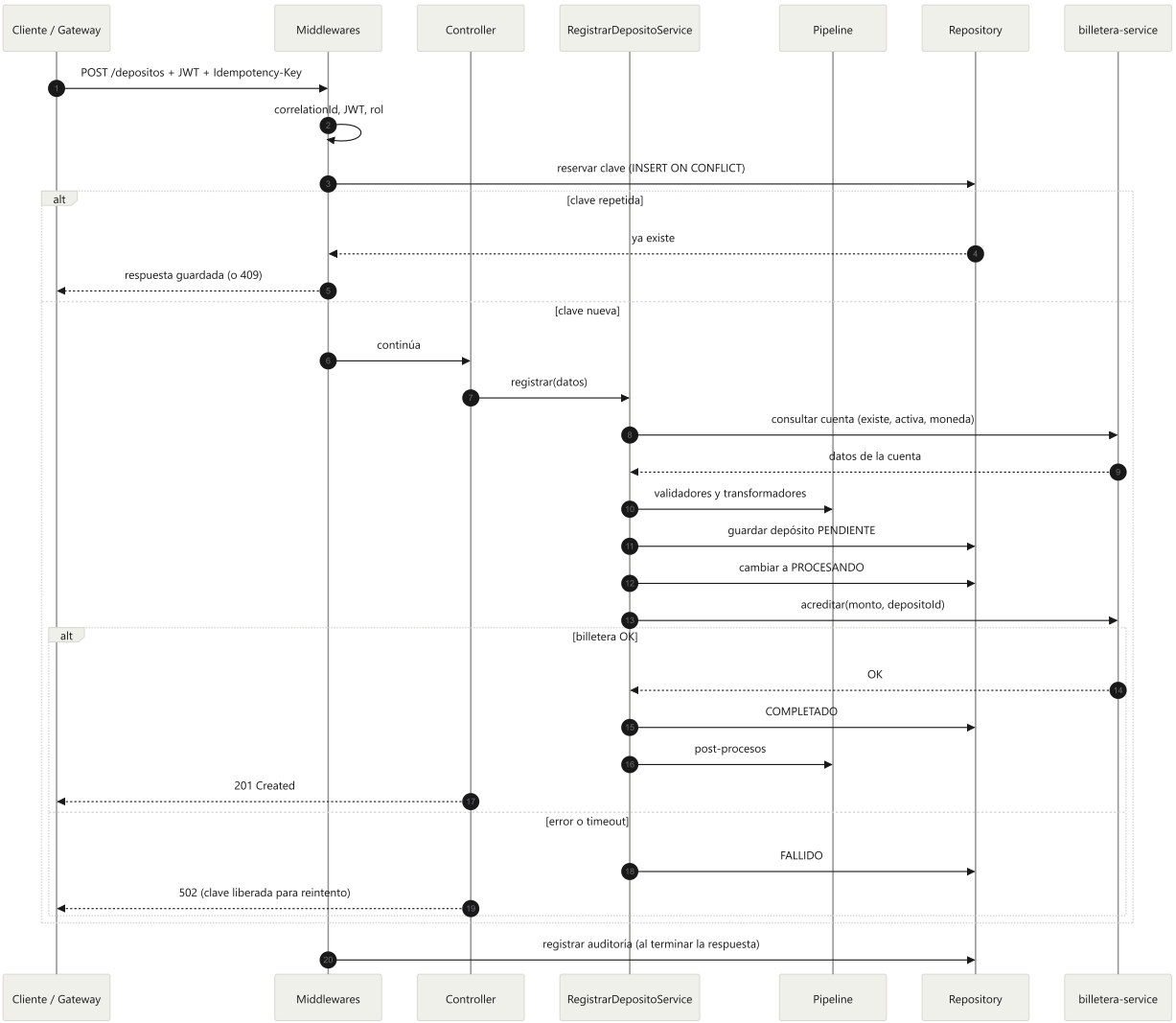


Contrato que debe cumplir cada módulo de extensión (`src/modules/<nombre>/index.js`):

```js
module.exports = {
  id: 'limites-por-canal',
  tipo: 'validador',          // validador | transformador | post-proceso
  orden: 10,                  // orden dentro del hueco
  ejecutar: async (deposito, contexto) => { /* ... */ },
};
```

Tipo	Qué puede hacer	Qué NO puede hacer
validador	Lanzar error de negocio (422)	Modificar el depósito
transformador	Modificar y devolver el depósito	Rechazarlo
post-proceso	Reaccionar al resultado	Cambiar el resultado ni la respuesta

4. Flujo del depósito (secuencia)



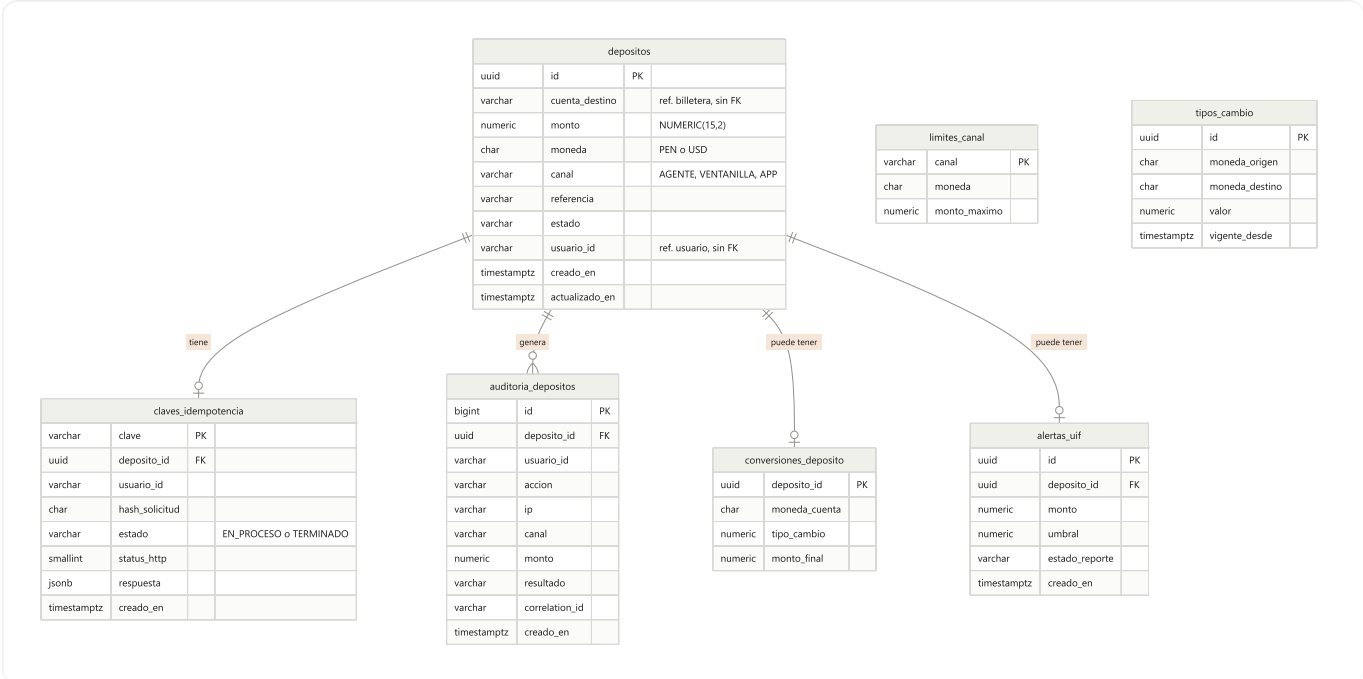
5. Estados del depósito



---

6. Modelo de datos

Cada módulo tiene su propia tabla con FK a `depositos`. `cuenta_destino` y `usuario_id` **no son FK**: pertenecen a billetera-service y usuario-service.



> `limites_canal` y `tipos_cambio` son tablas de configuración: los módulos las leen, no tienen relaciones.

7. Estructura de carpetas

```
deposito-service/
├── .claude/agents/
|   ├── desarrolladorback.md
|   ├── desarrolladorfront.md
|   ├── tester.md
|   └── revisor.md
├── specs/
|   ├── manifest.yaml
|   ├── 00-core/
|   ├── 10-transversales/   (seguridad, auditoria, idempotencia)
|   ├── 20-features/        (registrar, consultar, historial, limites,
conversion, uif)
|   └── 30-integraciones/   (billetera, usuario)
├── src/
|   ├── app.js
|   ├── core/
|   |   ├── db.js
|   |   ├── cargador.js
|   |   ├── pipeline.js
|   |   ├── errores.js
|   |   ├── middlewares/    (correlationId, auth, auditoria, idempotencia)
|   |   └── clients/        (billeteraClient.js)
|   └── modules/
|       ├── registrar-deposito/   (routes, controller, service, repository)
|       ├── consultar-deposito/
|       ├── historial-depositos/
|       ├── limites-por-canal/
|       ├── conversion-moneda/
|       └── alerta-uif/
├── db/                      (scripts SQL)
├── tests/
└── Dockerfile
```

8. Pendientes y decisiones

#	Tema	Estado	Notas
1	Endpoint de billetera para consultar cuenta	[POR ACORDAR]	
2	Endpoint de billetera para acreditar (idempotente con depositoId)	[POR ACORDAR]	
3	Algoritmo JWT, issuer, audience y formato de roles	[POR ACORDAR]	con usuario-service
4	Fuente del tipo de cambio	Pendiente	tabla de configuración o servicio externo
5	Umbral UIF vigente	Pendiente	verificar normativa SBS
6	Herramienta de API Gateway	Pendiente	ej. NGINX, Kong, Traefik
7	Tecnología del frontend	Pendiente	